BUDGER

Household Donut Privacy Bug Analysis

Root-cause investigation & full fix plan

July 2026 — Engineering Internal

Executive Summary

The household donut chart showed all member dashboards — including the head's own — as 'Private' after the head switched transaction categories several times. The underlying backend privacy logic is correct: the server never blocks the head from viewing anyone's data. The bug is entirely in the frontend: **any fetch error from *useGetMemberSpending* unconditionally triggers setIsPrivate(true)**, regardless of whether the error is a genuine privacy block (HTTP 403) or a transient server problem (500, timeout, network loss). Rapid category reassignment causes the server to run several expensive parallel DB queries simultaneously, which can return a 500 error — falsely locking every drilled member view as 'Private'.

Six distinct failure scenarios have been identified. Each is described below with its trigger, root cause, user impact, and the specific code change needed to resolve it.

Severity legend

CRITICAL	Directly reproducible; corrupts core UX, destroys user trust.	HIGH	Reproducible under common usage; significant UX damage.
MEDIUM	Uncommon or requires edge conditions; noticeable but recoverable.	LOW	Rare or cosmetic; polishes reliability.

Failure Scenarios

Scenario 1	Any fetch error treated as a privacy block	CRITICAL
Location	frontend · HouseholdDonutChart.tsx line 547–549	
Trigger	The useGetMemberSpending query returns any non-2xx response while the donut is in drill-down mode (drillPhase ≠ 'idle'). Caused here by rapid category switching which temporarily overloads the server.	
Root cause	useEffect(() => { if (memberSpendError && drillPhase !== 'idle') setIsPrivate(true); }) does not inspect the HTTP status code. A 500, 503, network timeout, or any other transient error sets isPrivate = true identically to a genuine 403 'blocked' response. The head's own donut is also affected because the frontend has no self-exemption — only the backend does.	
Impact	All drilled member views, including the head's own dashboard, lock to the 'Private' padlock screen for the rest of the session. The user must close and reopen the app to recover. Completely destroys trust in the privacy system.	
Fix summary	Inspect the HTTP status code of memberSpendError. Only set isPrivate(true) when the status is exactly 403. For all other errors show a dismissible error banner with a Retry button instead of the padlock animation.	

```
// Change the effect to check error.status before locking: useEffect(() => { if (!memberSpendError || drillPhase === 'idle') return; const status = (memberSpendError as any)?.status as number | undefined; if (status === 403) setIsPrivate(true); else setMemberFetchError(true); // new state → show retry banner }, [memberSpendError, drillPhase]);
```

Scenario 2	Server overload from concurrent category-switch invalidations	HIGH
Location	backend · households.ts GET /households/members/:userId/spending	
Trigger	The head reassigns transaction categories several times in quick succession. Each save fires multiple queryClient.invalidateQueries calls; the donut's useGetMemberSpending query refetches concurrently with these.	
Root cause	The member spending endpoint fires five parallel DB queries (transactions, categories, recurringPayments, rpLogs, householdMember). Under concurrent load from rapid category mutations, one of these may time out or the Neon connection pool may be exhausted, causing a 500 response. Combined with Scenario 1, this is sufficient to trigger the privacy lockout.	
Impact	Intermittent 500 errors that the frontend misreads as privacy blocks. Also degrades perceived backend reliability. Likely the direct cause of the reported crash.	
Fix summary	1) getGetMemberSpendingQueryKey is not in the category-change invalidation list — confirm and document this explicitly so future changes do not accidentally add it. 2) Add a try/catch around the parallel DB block in the spending route and return 503 with a machine-readable code ('db_error') instead of letting Express throw a 500. 3) After Scenario 1 is fixed, a 503 will show a retry banner rather than a padlock.	

Scenario 3	isPrivate state not reset when drilling back	HIGH
Location	frontend · HouseholdDonutChart.tsx startDrillBack() ~line 782	
Trigger	A member's donut shows the privacy padlock (isPrivate = true, from any previous error or a real 403). The user taps to drill back to the household view. They then tap-and-hold to drill into a different member.	
Root cause	startDrillBack() never calls setIsPrivate(false). The isPrivate state persists across the drill-back animation. When startDrillDown() fires for the next member it does call setIsPrivate(false), but there is a React render cycle between removeQueries and the new fetch completing. If the new member's query also has a cached error at that moment, the useEffect fires again before setIsPrivate(false) is processed.	
Impact	A padlock shown for member A can bleed visually into the drill-back or the next member's view. In the worst case the lock animation re-triggers immediately for a member who is not actually private.	
Fix summary	Add setIsPrivate(false) and setLockPhase(null) at the very start of startDrillBack(), before any animation timers are scheduled. This ensures the lock state is always cleared the moment the user begins retreating from a drilled view.	

```
function startDrillBack() { if (drillPhase !== 'personal') return;
lockTimersRef.current.forEach(clearTimeout); lockTimersRef.current = []; setIsPrivate(false); //
← add this setLockPhase(null); // ← add this // ... rest of existing drill-back logic }
```

Scenario 4	Window-focus refetch triggers error on iOS PWA return	HIGH
Location	frontend · HouseholdDonutChart.tsx useGetMemberSpending options	
Trigger	The user drills into a member's donut, switches to another iOS app (banking app, messages, etc.) and returns to Budger. React Query's default refetchOnWindowFocus fires the member spending query. If the server is briefly slow at that moment (wake-up latency on Render's free tier, Neon cold start), the fetch errors.	
Root cause	useGetMemberSpending has no refetchOnWindowFocus override, so it inherits React Query's global default of true. Combined with Scenario 1's error-to-private mapping, a single focus event can lock all member views.	
Impact	Especially damaging on iOS where app-switching is the primary multitasking model. Users who briefly check another app and return find their household donut locked.	
Fix summary	Set refetchOnWindowFocus: false on the useGetMemberSpending call inside HouseholdDonutChart. Member spending data does not need to be instantly fresh on focus return; it refreshes naturally when the user drills back and re-drills. Alternatively, set a staleTime of at least 60 seconds.	

```
const { data: memberSpendRaw, isError: memberSpendError } = useGetMemberSpending(realMemberId, {
query: { enabled: drillPhase !== 'idle' && !isVirtualDrill && realMemberId > 0,
refetchOnWindowFocus: false, // ← add staleTime: 60_000, // ← add retry: false, // ← add (avoids
retry storms) }, });
```

Scenario 5	Household membership drift causes 403 on self or 404 on valid member	MEDIUM
Location	backend · households.ts GET /households/members/:userId/spending line 343	
Trigger	A user leaves and rejoins a household (or is removed and re-invited). In rare cases the users.householdId column and the household_members table become temporarily inconsistent — the backend code itself notes this risk in a comment. Can also happen if a member is removed while someone else is drilling into their donut.	
Root cause	The route checks currentUser.householdId (from the users table) but looks up membership by userId alone in household_members. If these are out of sync, the route returns 403 ('Not in a household') for currentUser or 404 ('Member not found') for targetUser. Frontend treats both as an error → Scenario 1 fires → privacy lockout.	
Impact	Intermittent: only affects users who recently changed household membership. They see all member donuts as 'Private' until app reload syncs the cache.	
Fix summary	1) Use machine-readable error codes for 403/404 ('not_in_household', 'member_not_found') so the frontend can distinguish them from a real privacy block. 2) After Scenario 1 is fixed these will show a retry banner rather than a padlock. 3) On the backend, consider a single source of truth query that joins users and household_members rather than checking both independently.	

Scenario 6	React Query retry storms re-trigger the lock animation	MEDIUM
Location	frontend · HouseholdDonutChart.tsx useGetMemberSpending + lock animation effect	
Trigger	Any transient error on the member spending fetch. React Query retries failed queries up to 3 times by default with exponential backoff.	
Root cause	Each retry cycle that fails sets memberSpendError = true then false then true again. The useEffect on [memberSpendError, drillPhase] fires on every transition. Combined with Scenario 1 this re-triggers setIsPrivate(true) and the lock animation starts over multiple times — padlock pops, fades, then pops again — deeply confusing UX even if only a transient server hiccup occurred.	
Impact	Visual glitches: the padlock animation plays 2–3 times. After retries succeed (if the error was transient), isPrivate may remain true because setIsPrivate(false) is never called on query recovery.	
Fix summary	Set retry: false on useGetMemberSpending (covered in Scenario 4's fix). Additionally, add a recovery path: if memberSpendError was true but the query subsequently succeeds (isSuccess flips to true), call setIsPrivate(false) and setMemberFetchError(false) to restore the normal view automatically.	

```
useEffect(() => { // Clear error states if a previously-failed query recovers if
(!memberSpendError && drillPhase !== 'idle') { setMemberFetchError(false); // Do NOT auto-clear
isPrivate — a real 403 should stay locked // until the user explicitly drills back out. } },
[memberSpendError, drillPhase]);
```

Implementation Order

All six scenarios should be resolved together in a single focused sprint. The table below shows the recommended implementation order — earlier items unblock later ones.

1	Scenario 1: only 403 → isPrivate	HouseholdDonutChart.tsx	All others	30 min

2	Scenario 4: disable refetchOnWindowFocus + retry:false	HouseholdDonutChart.tsx	4, 6	10 min
3	Scenario 3: reset isPrivate in startDrillBack	HouseholdDonutChart.tsx	3	10 min
4	Scenario 6: add recovery path on query success	HouseholdDonutChart.tsx	6	15 min
5	Scenario 2: add try/catch + 503 in spending route	households.ts	2	20 min
6	Scenario 5: machine-readable error codes on 403/404	households.ts	5	15 min

Verification Checklist

After all fixes are applied, run through each of the following test cases before shipping:

1	Drill into any member; rapidly switch categories 5+ times on the Transactions page while staying drilled. Donut must NOT show padlock.	CRITICAL
2	Drill into own donut as head. Must never show padlock (own data is never blocked).	CRITICAL
3	Simulate a 500 from /members/:id/spending (e.g. via DevTools throttle → Offline). Must show retry banner, not padlock.	CRITICAL
4	Drill into a member, switch apps for 5 s, return to Budger. Donut must still show data, not padlock.	HIGH
5	Drill into member A (with real 403 = private). Drill back. Drill into member B (not private). B must show real data.	HIGH
6	Drill into a private member. Drill back. Verify isPrivate is reset (household view looks normal).	HIGH
7	Trigger a transient 503 during drill. Verify no padlock, error banner shows. Restore connectivity; verify retry auto-refreshes data.	MEDIUM
8	Remove a member while another user's session is drilling into that member. Must show 'member not found' banner, not padlock.	MEDIUM

This document covers only the household donut privacy system. No other subsystems (Larder, Goals, Splits, Recurring Payments) are in scope.